

Multi-Agent Workflow & Prompt Engineering

Vibe Engineering for AI-Generated Systems (No-Code Focus)

קורס מטמיעי ומיישמי AI בארגונים



למה אנחנו בכלל כאן?

AI מייצר קוד

הכלים נהיו חזקים

ארגונים רוצים מהירות

אבל רוב המערכות נשברות אחרי שבוע

מה ההבדל בין Chat ל-System?

Chat:

- שאלה → תשובה
- אין state אמיתי
- אין validation
- אין audit trail

System:

- Intent
- תכנון
- חלוקת אחריות
- בדיקה
- Governance
- Traceability

Multi-Agent Architecture

חלק 1

עם עומק והסבר

למה Single Agent לא עובד ב-Scale?

ערבוב חשיבה וביצוע

אין ביקורת פנימית

אין עצירה ל-validation-

זיהום קונטקסט

קשה לדיבוג

לכן Human in the Loop: הוא חובה, לא המלצה



מה זה Multi-Agent באמת?

Context מוגבל לכל Agent

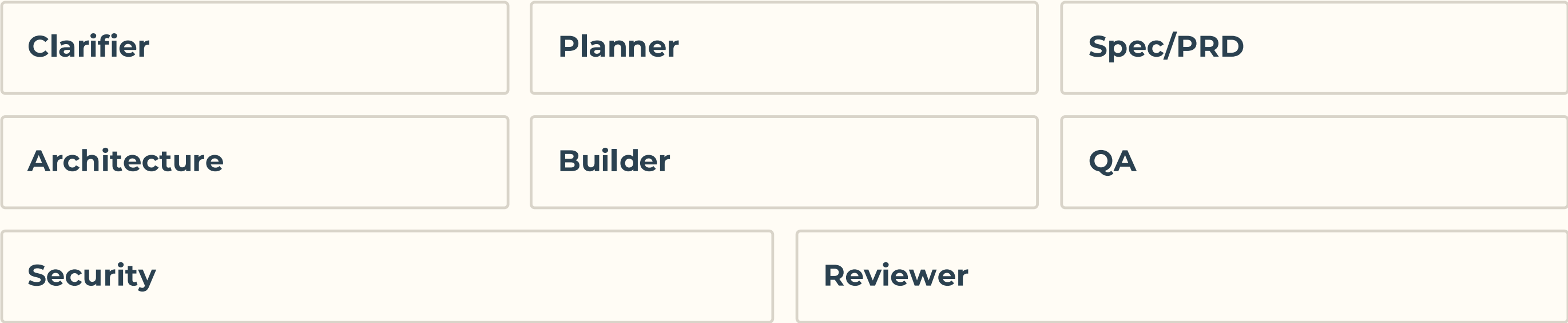
חלוקת אחריות

Orchestrator שמנהל את כולם

Output קשיח

תכנון Agents (עם הסבר)

Agents בסיסיים:



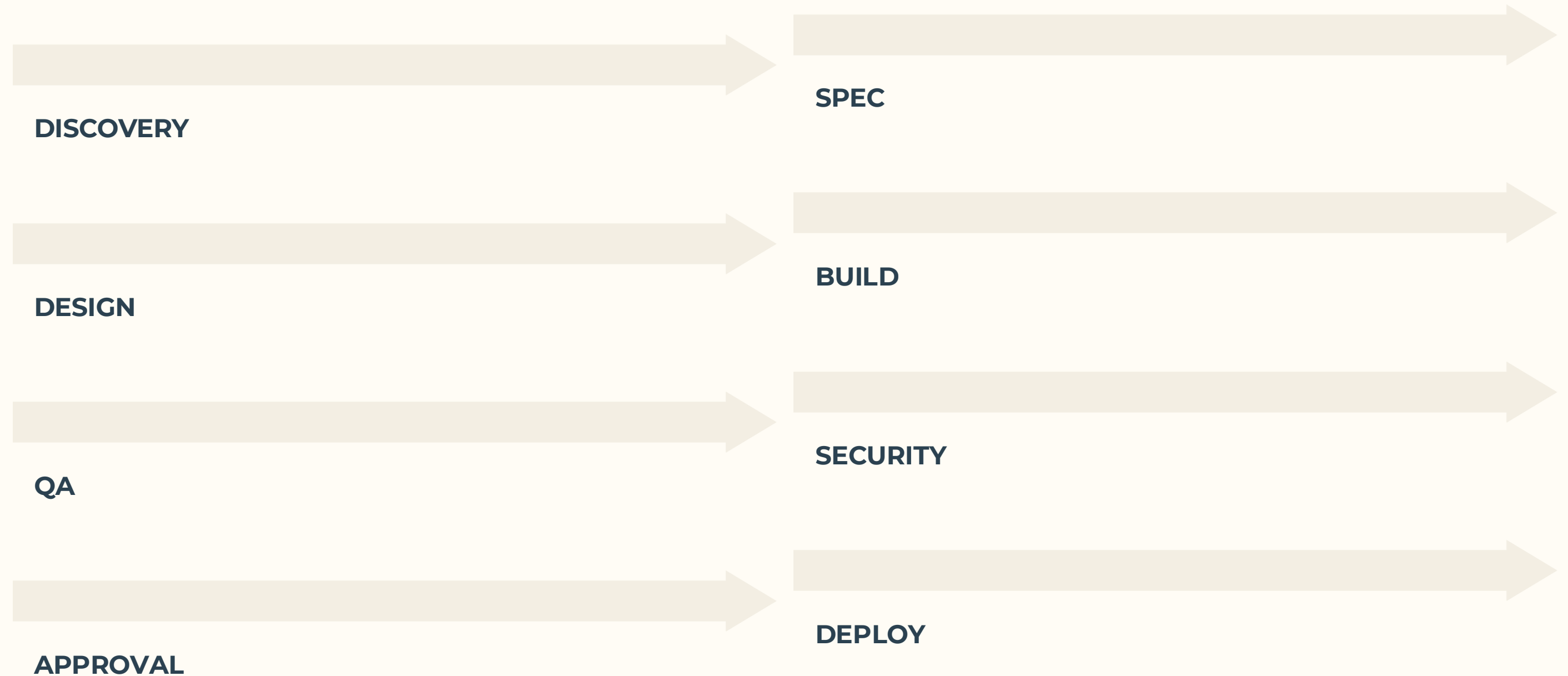
Orchestrator עמוד

Orchestrator אחראי על:

סדר הריצה	ניהול State
Retry strategy	Validation
Human gates	Confidence thresholds

State Machine Thinking

Workflow אמיתי נראה כך:



עמוקיםFailure Modes

Recursive refinement

Architecture drift

Over-generation

Context overflow

Agent dominance

Advanced Prompt Engineering

בעיות מורכבות כוללות:

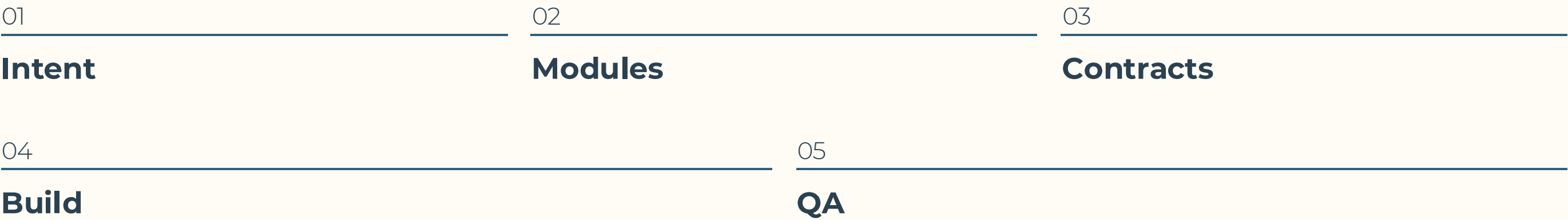
חשיבה	תכנון
כתיבה	ביקורת
בדיקה	אינטגרציה

Prompt הוא חוזה

Prompt כולל:

משימה	תפקיד
סכימת פלט	אילוצים
מדיניות הבהרה	כלל אימות

Layered Prompting



Output Schema

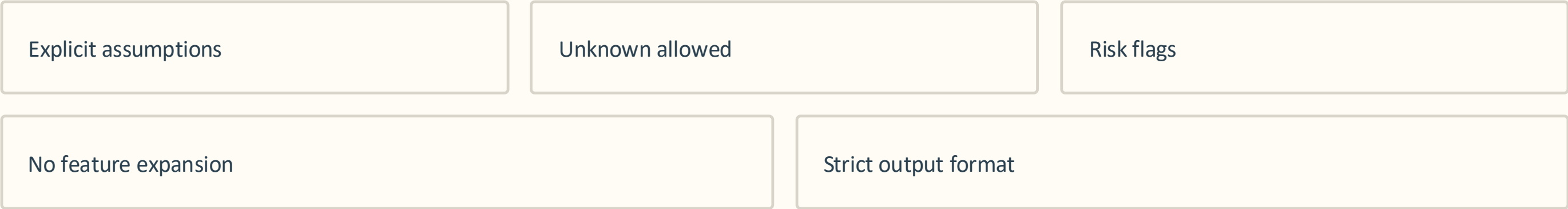
למה חשוב?

מאפשר Retry מדויק

מאפשר השוואת גרסאות

מאפשר בדיקה אוטומטית

Anti-Hallucination Techniques



Temperature Strategy

→ Planner נמוך

→ QA נמוך מאוד

→ Brainstorm גבוה

→ Security נמוך

Context Hygiene

יותר ≠ Context יותר טוב

Noise

Conflicts

Token explosion

Latency

Production LLM Thinking

Cost Modeling

שאלות שחייבים לשאול:

- כמה tokens לכל שלב?
- כמה retries?
- איזה מודל לכל Agent?
- כמה workflows ביום?

Latency Tradeoffs

-יותר → Agents יותר יציבות → יותר זמן

-יותר → Validation יותר בטיחות → יותר עיכוב



אין פתרון מושלם.

יש בחירות מודעות.

Model Routing

לא כל Agent צריך GPT-4.

Clarifier

מודל קל

QA

בינוני

Architecture

חזק

Drift Detection

עם הזמן:

פתרון:

בעיות:

- PRD משתנה
- Scope
- מערכת נהיית לא עקבית

- Snapshot comparison
- חוזרים ל PRD-Anchor
- Regression artifact check

Vibe Coding ללא קוד

חזק ב:

- Prototyping
- מהירות
- Non-developers

סיכון:

- Over-abstraction
- Lock-in
- שליטה נמוכה

Architecture Collapse

כלי AI builder נכשלים:

ליצור תלותיות

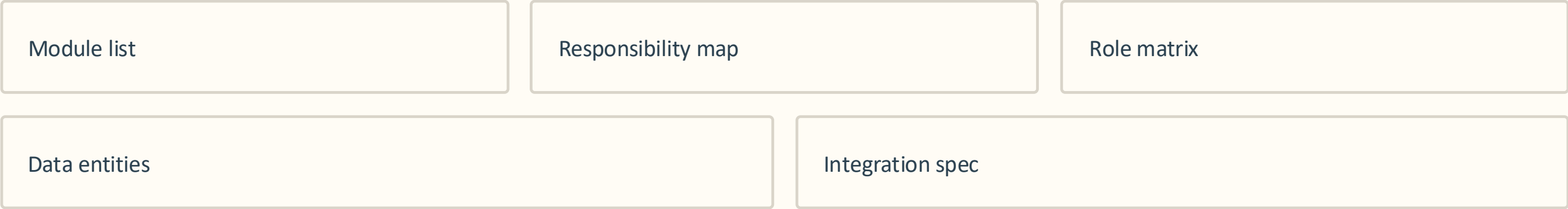
למזג מודולים

לשבץ boundaries

להוסיף פיצ'רים

Contract-First Build

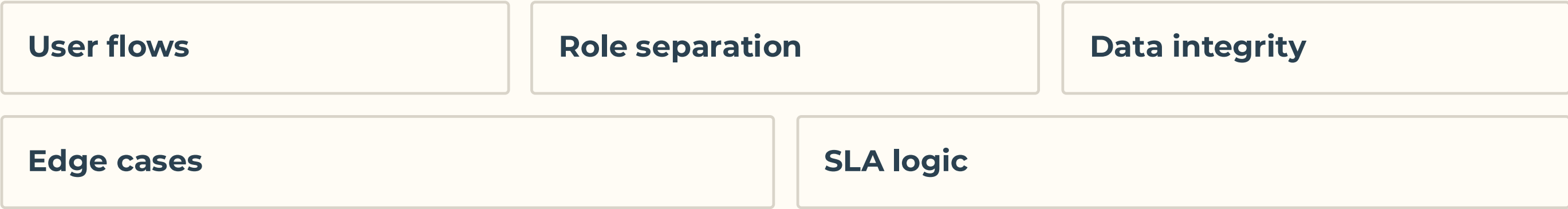
לפני build:



רק אז builder.

QA ללא קוד

בודקים:



QA הוא חשיבה מערכתית, לא קריאת קוד.

AI בלי טסטים = חוב עתידי



Security by Design (No-Code)

בלי להניח. תמיד לבדוק.

Authentication?

Authorization?

Admin isolation?

Audit logs?

Secrets?

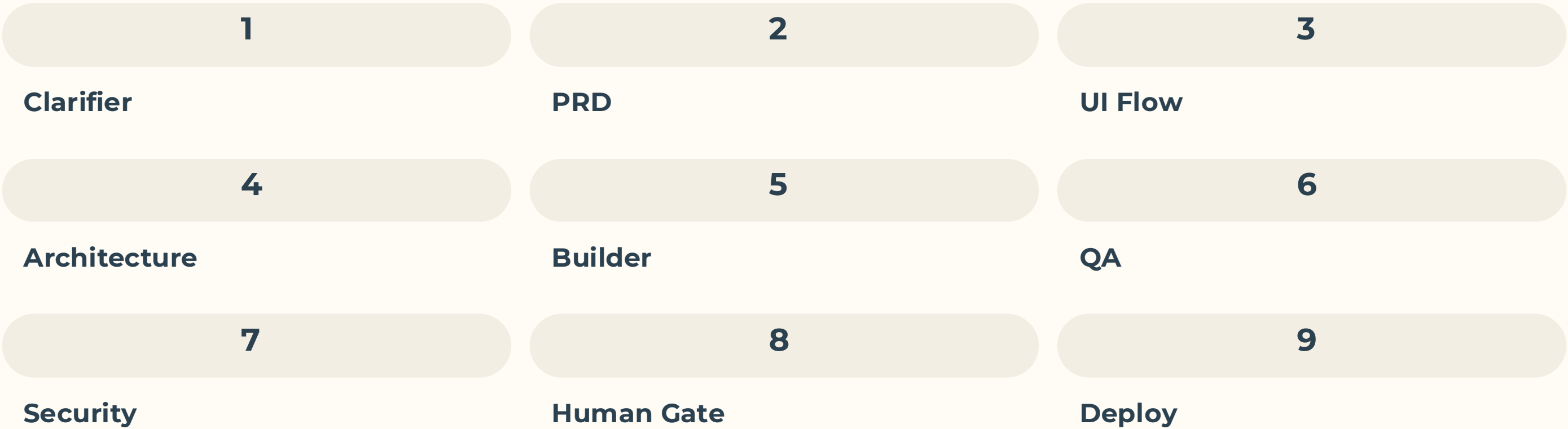
Rate limits?

Ticketing SaaS (No-Code)

בעיה:

קריאות במייל
אין
אין סטטוס

Workflow:



Mermaid Diagram

```
graph TD
    A[Intent: Ticketing SaaS] --> B[Clarifier Agent]
    B --> C[PRD Agent]
    C --> D[UI Flow Agent]
    D --> E[Architecture Agent]
    E --> F[AI Builder Tool]
    F --> G[QA Agent]
    G --> H[Security Agent]
    H --> I{Human Approval}
    I -->|Approve| J[Deploy]
    I -->|Reject| C
```

מה למדתם באמת

Multi-Agent = ארכיטקטורה

Prompt = חוזה

Workflow = State machine

No-Code דורש יותר הנדסה

Human Gate = בטיחות

Governance = Scale

המשפט האחרון

AI מייצר מהר. הנדסה מייצרת
— נכון. מי שמשלב את שניהם
בונה מערכות אמיתיות.